Go Statement Considered Harmful (2018) (vorpus.org)

176 points by celeritascelery on March 19, 2021 | hide | past | favorite | 82 comments

orisho on March 19, 2021 | next [–]

There's one key difference between Go and all the other examples: an uncaught panic in any goroutine, not just the main goroutine, crashes the app.

Often your main goroutine will just be starting other goroutines, in a multiple-services-one-process architecture, and it doesn't care whether they succeed or not, and its code need not be aware of whether they're critical. If they are, they should panic and crash the program. Just like a main function that crashes.

Doing things with the nursery is restrictive which has its benefits, but I don't agree that it's just strictly always the preferred approach. I prefer an approach like Go's, where you can choose whether to use nurseries (errgroups), and the implementation for them is provided by the language authors. Sure, you are able to start and manage goroutines in this way, even if you don't care about their lifetime - as they are effectively individual programs, but you shouldn't HAVE to.

So when you care about goroutines running only while their starter goroutine is at a certain scope, you can do so. But you don't have to, and when you don't need it - you don't need to pay the price. Go appears to take an approach that provides primitives that are as least restrictive as possible to make it easy to read code, but then also has a batteries included approach with its libraries.

d0mine on March 19, 2021 | parent | next [–]

Yes, the "nurseries" approach introduces restrictions compared to the "go" construct in the same way as a "for" loop with break/continue is more restrictive compared to "go to." The title of the article strongly hints at it too.

Nullabillity on March 19, 2021 | parent | prev | next [–]

Neither model is more powerful than the other, they can both emulate the other (errgroups to bolt some structure onto unstructured concurrency, global nurseries to leak tasks from the structure).

The difference is which option they encourage, and which option requires an explicit choice.

bccdee on March 19, 2021 | parent | prev | next [–]

It seems like the article is, at its most basic, just arguing in favour of using synchronization primitives. Which, yeah, of course. Most of the time, you want to be doing that; I don't think that's controversial. The author likes scoped primitives best, and sure, fair enough.

But it takes a couple strange turns when it suggests that (a) you should ALWAYS use sync primitives, and (b) waitgroups/nurseries are the only sync primitive worth using.

If I'm only spawning off one parallel thread, a simple join statement is all I need. If I'm doing an async task, a promise is plenty.

If there's a true fork in my execution, maybe I don't want to use a sync primitive at all. Go cleans up stray goroutines on process termination, so if my program's logic doesn't demand that my goroutines join back together, why should they? Let them terminate themselves, or let them live forever.

There are more concurrency models in heaven and earth than are dreamt of in this guy's philosophy. It's not that I think waitgroups are a bad primitive or anything, I just think it's a bit much to take a useful primitive and go "this should be the only tool that is ever used to manage concurrency."

egeozcan on March 19, 2021 | parent | prev | next [–]

I have little experience in go, and didn't know about errgroups, thanks! Are "golang.org/x" packages something like an extended standard lib?

https://pkg.go.dev/golang.org/x/sync/errgroup

evil-olive on March 19, 2021 | root | parent | next [–]

Yes, golang.org/x/ is of a "nursery" of sorts for things that are stdlib-ish but not fully baked enough that the maintainers want to support it under the 1.0 compatibility promise [0]

https://golang.org/pkg/#subrepo

> These packages are part of the Go Project but outside the main Go tree. They are developed under looser compatibility requirements than the Go core.

0: https://golang.org/doc/go1compat

eqvinox on March 19, 2021 | parent | prev | next [–]

A SIGSEGV or SIGABRT (e.g. assert() failure) in any thread in C/C++ code will also crash the entire process.

pjmlp on March 19, 2021 | root | parent | next [–]

Not if there is an handler configured, although it is UB what happens next.

eqvinox on March 19, 2021 | root | parent | next [–]

Sure you can install a handler to get another outcome, just like you can catch exceptions/panics/... in additional threads in languages that have these constructs. But this was about "uncaught panic", i.e. default behavior without handlers.

(Also, SEGVs are well behaved enough to implement userspace virtual memory on top of, if that's your fetish... not sure what the spec says, but it's not UB in actual practice. Trying to resume from an assert() is indeed UB though since that'd be returning from a noreturn function.)

pjmlp on March 19, 2021 | root | parent | next [–]

It is POSIX UB, as the standard gives no guarantees what happens when the signal returns in such cases, so each OS is free to do whatever they feel like.

wahern on March 19, 2021 | root | parent | next [–]

You don't have to return from the signal handler. You could also longjmp. longjmp happens to be async-signal-safe, and the standard is careful to permit longjmp out of a signal handler, so long as you're careful to avoid implicating other non-safe behaviors. SIGSEGV is a synchronous signal (that is, always delivered immediately to the same thread that triggered it), and presumably the point at which SIGSEGV is triggered was carefully orchestrated by the application. I've done this once--it permitted me to remove a boundary check on an array, improving runtime performance many fold in a machine generated, statically compiled NFA. On SIGSEGV I would longjmp back to a safe entry point, grow the array, and then restart the NFA.

pjmlp on March 19, 2021 | root | parent | next [–]

Did you tried across several POSIX platforms?

wahern on March 20, 2021 | root | parent | next [–]

It was developed on macOS and deployed on Linux/glibc. I've also longjmp'd out of signal handlers to emulate sigtimedwait on OpenBSD and NetBSD . I *probably* (can't remember) also tested the sigtimedwait implementation on FreeBSD, Linux, and Solaris (and maybe AIX), my typical porting targets.

_ktx2 on March 19, 2021 | parent | prev | next [–]

I don't think that's true. A panic in a goroutine should shut down the goroutine but not the main thread.

Did this change at some point?

hexmiles on March 19, 2021 | root | parent | next [–]

if not catched in a recover() a panic when reach the top of the stack of the gorutine always cause a crash. I did not changed, as far i know is this way at least since 1.0

_ktx2 on March 19, 2021 | root | parent | next [–]

https://goplay.tools/snippet/am2vcq5JYFr

This is an example I made. Can you show how that would change to cause the problem you're describing?

alex_studer on March 19, 2021 | root | parent | next [–]

It looks like maybe the goroutine in that example doesn't actually get a chance to run before the program stops. If the main function lasts longer, then you'll see the effect of the panic. See https://goplay.tools/snippet/8SFlFkZ2P0y

Also, see the Go spec: https://golang.org/ref/spec#Handling_panics

> Next, any deferred functions run by F's caller are run, and so on up to any deferred by the top-level function in the executing goroutine. At that point, the program is terminated and the error condition is reported, including the value of the argument to panic.

It looks like, assuming it makes it to the top of the current goroutine, then it should be killing the whole program.

_ktx2 on March 19, 2021 | root | parent | next [–]

Yeah, very interesting. it seems highly inconsistent even when using a waitgroup: https://goplay.tools/snippet/X2c7lZGTqww

Personally I don't use panics or deal in them that much, mainly because stack traces with multiple goroutines are unbelievable. I much prefer errors.

GauntletWizard on March 20, 2021 | root | parent | next [–]

Your use of the waitgroup is off; Because the Add instruction is inside the goroutine, it may not run until after wg.Done is called, which will return immediately.

Still, that made it less flaky for a bit, but the it printed again! And I realized what else is going on: There's actually time between when the deferred functions are executed and the program exits! Which is fascinating, but unlikely to matter in practice. It brings me to another question: do deferred statements still get called when there's a panic on a *different* goroutine? And the answer seems to be no; adding a deferred print to the main function does not print.

_ktx2 on March 21, 2021 | root | parent | next [–]

You're right about the Add(1) and defer. defer in itself is pretty interesting in how it's called. Basically there's the main body of your function and then a list of things to do *before return*, of which defers get added to. So, your theory is correct.

I think intuitively I probably knew that a panic in a goroutine will shut down the main thread, it just didn't occur logically to me when I read it; which is an interesting paradox. Maybe that's a me-thing though.

zzbzq on March 19, 2021 | prev | next [–]

Dijkstra's paper is hard to read. I've gone through it and broken it down. It turns out, the central argument put forth in Dijkstra's paper is that the 'goto' statement messes up the stack trace. Although I don't think the term "stack trace" existed yet, so he keeps talking about "textual indices."

The 'go' statement actually allows the cleanest stack traces of any language I've encountered. Go crash dumps can print 40 stack traces for 40 threads and still be shorter, cleaner, easier to read than the stack traces of something like a single line of C# async/await code.

The article seems to argue that it's hard to run more code after the completion of a goroutine. But, the entire point is you don't have to. Go does async i/o under the hood, so you just write the lines of code sequentially in the order you want them to happen, and they will happen at the right time.

As a side note about the 'goto' statement, modern languages, if they have the statement at all, explicitly lock 'goto' usage into one stack frame anyway. I'm 100% unconvinced there's anything wrong with it in those instances. Anything bad things you can do with it can be done wrong as easily with other control flow.

goto11 on March 19, 2021 | parent | next [–]

Dijkstra is not talking about stack traces. Structured programming is not just about functions, it is also control structure blocks like if or while-blocks instead of goto's, which does not affect stack traces.

Dijkstra is talking about describing and analyzing the execution of a program. With structured programming, the execution of a program can be described as a tree of executed blocks. A program using goto's is a tangled graph, since every point can jump to any other point. This makes it much harder to describe and analyze.

I doubt Dijkstra cared much about stack traces, since he didn't care for debugging at all. For him, programs was something you analyzed on paper and proved correct.

As for gotos being constrained to a single function, this does not make any difference for Dijkstras point, since a single function can still contain the same complexity as a whole unstructured program.

Banana699 on March 19, 2021 | root | parent | next [–]

(Imperative) Programs cannot be pure trees, looping constructs necessarily introduce cycles. What structured programming is all about is to make those cycles as well-behaved as possible.

You are largely right nonetheless, Dijkstra's argument could be summarized as:

1- Understanding code depends on imagining it's dynamic effects from its static description

2- As we progressively make code more powerful (executing single statements in sequence -->+ executing either of two statements depending on the current state -->+ aggregation of statement blocks into subroutines -->+ looping constructs); the job of the entity reading your code becomes progressively harder as it tries to reason about: a List -->+ a Tree -->+ a generalized extra-recursive tree where nodes are themselves trees -->+ that last kind of tree but with cycles.

3-GoTos create especially-nasty uber-unpredictable networks of cycles while adding no more power than other well-behaved options.

The essence of the argument is a tale as old as time: increasing the Freedom of Expression for the writer increases the Burden of Understanding on the reader.

Stack traces, in a generalized kind of way, is another equivalent form of describing Dijkstra's argument. The previous way of explaining Dijkstra's argument imagines an idealized reader (could be an actual person, a static-analysis tool trying to predict some property of your code, or any other "reader") and asks you to consider the complexity of the data structure it tries to construct to mentally represent your program in static form.

Now imagine it the other way: the reader is seeing your program's dynamical behavior in a "debugger" (could be an actual debugger, could be a sheet of a paper with a bunch of trace tables,...), how does the amount and complexity of the data structures that this idealized reader needs to follow the behaviour of your code scales as the complexity of your code scales? For purely sequential programs , that 's just a single counter keeping track of where we're in the program . Add conditionals, and you now have to keep track of a flag indicating for each line which branch was taken. Add blocks or subroutines , and you are now maintaining a whole stack of such counters/flags.

So far, so linear, you are still limited to incrementing counters, raising flags, and adding or removing counters/flags from the top of the stack. But you're never decrementing counters (going back to a previous point in execution) or touching the stack in the middle. Gotos and loops destroy this: Gotos are the worst of the pair as it arbitrarily jumps you to any counter on the whole stack without even recording the point of return. The whole concept of an "execution stack" becomes meaningless, program execution is a graph that you maintain in your head (or in the debugger) as it threads through the code. Other looping constructs are more well-behaved in that they state beforehand that a jump will happen and also the fixed whereabouts of its destination. The argument is that since different constructs give you the equivalent power but complicate your code's dynamic behavior asymmetrically, you should choose some of them and ban others.

The first version of the argument imagines code understanding as a writer-reader dynamic and talks about the reader reasoning about the static form of the programs written by the writer, the second imagines it as a dancer-viewer dynamic and talks about the viewer following the flow of the program orchestrated by the dance choreographer (the programmer). Two ways of saying the same thing.

curryst on March 19, 2021 | parent | prev | next [–]

> The article seems to argue that it's hard to run more code after the completion of a goroutine. But, the entire point is you don't have to. Go does async i/o under the hood, so you just write the lines of code sequentially in the order you want them to happen, and they will happen at the right time.

That seems a little hard to believe. If I do this:

```
db, _ := sql.Dial("")
defer db.Close()
go myFunc(db)
```

It will close my DB connection while myFunc is still running (assuming myFunc isn't trivial enough that it sometimes runs before the close can happen).

The author seems like they were more concerned about how 'go' tends to break abstractions. The additional flow control means that I do actually have to care how the library is implemented. I have to understand how they handle the lifecycle of objects I pass in, and then I have to figure out how to stop their stuff (and to stop it at the right time). DB connections are one of the things in Go that are frequently not closed or not closed correctly. The lack of a "with" kind of statement requires me to write a bunch of IPC so I can tell goroutines to quit, then wait for them to quit, and then finally close the connection. A lot of people (myself included when I'm being a bad programmer) just call 'os.Exit' and figure the DB will clean up the stale connections.

mykowebhn on March 19, 2021 | root | parent | next [–]

Using a waitgroup will fix your problem in your example.

cratermoon on March 19, 2021 | root | parent | next [–]

Go convention would be to pass a 'done' channel to myFunc and only close the db on receipt of a message on the channel.

Jtsummers on March 19, 2021 | parent | prev | next [–]

https://www.cs.utexas.edu/users/EWD/transcriptions/EWD02xx/E... (for a reference for others)

Your statement:

> It turns out, the central argument put forth in Dijkstra's paper is that the 'goto' statement messes up the stack trace.

Didn't sit right with my recollection so I reread it, it's brief so I recommend others take the time to read it themselves (I think it's one of those short and easy texts that the vast majority people have heard of and reference without ever having read, so be the change and read it if you haven't).

His central thesis is laid out in paragraphs two and three, with the rest being an attempt at explaining why the go to statement creates problems and his suggested improvement:

> My first remark is that, although the programmer's activity ends when he has constructed a correct program, the process taking place under control of his program is the true subject matter of his activity, for it is this process that has to accomplish the desired effect; it is this process that in its dynamic behavior has to satisfy the desired specifications. Yet, once the program has been made, the "making' of the corresponding process is delegated to the machine.

> My second remark is that our intellectual powers are rather geared to master static relations and that our powers to visualize processes evolving in time are relatively poorly developed. For that reason we should do (as wise programmers aware of our limitations) our utmost to shorten the conceptual gap between the static program and the dynamic process, to make the correspondence between the program (spread out in text space) and the process (spread out in time) as trivial as possible.

In summary, shortening the semantic distance between the static representation and the dynamic execution improves our ability to understand that dynamic execution. And that is critical because it's the actual thing we, as programmers, are producing.

dang on March 19, 2021 | prev | next [–]

Discussed at the time:

Notes on structured concurrency, or: Go statement considered harmful - https://news.ycombinator.com/item?id=16921761 - April 2018 (230 comments)

dnautics on March 19, 2021 | prev | next [–]

The erlang case suffers from none of these problems, plus it's even more sophisticated, you can set up failure domains with few lines of code; it's designed for the spawn to be truly autonomous with restart/retry logic, do rpc over distributed systems with unreliable networks, etc.

Once you've spent time working in erlang concurrency, it's hard to believe that this pattern is an advantage except for the fact that python (and go, and js) is designed for linear primary execution flow. In erlang by contrast, a script with a primary flow is a special case of a concurrent system (which itself is a special case of a distributed system).

Addendum: If I'm not mistaken, the nursery concept does exist in the BEAM vm, it's Task.await_many in elixir.

kasperni on March 19, 2021 | prev | next [–]

The wikipedia entry for structured concurrency provides a good intro into the subject [1].

Java in process of adding support for it as part of project Loom[2]. Kotlin has already based their coroutine support on it [3]. It is also on the roadmap of Swift [4].

[1] https://en.wikipedia.org/wiki/Structured_concurrency

[2] https://wiki.openjdk.java.net/display/loom/Structured+Concur...

[3] https://elizarov.medium.com/structured-concurrency-722d765aa...

[4] https://forums.swift.org/t/swift-concurrency-roadmap/41611

vlaxer on March 19, 2021 | parent | next [–]

The Wikipedia entry is pretty low on facts and historical context. In the previous discussions many people have disputed the originality of the approach that the Wikipedia entry seems to claim.

lacker on March 19, 2021 | prev | next [–]

So here's an example that I'm not sure how it would work with nurseries. Let's say I'm writing some sort of logging functionality for a larger codebase. The API in my library is `log(message)` where message is some random message object. It just appends the message to a file, but if the file size has gotten large enough then it does some slow work, maybe it batches things up and sends them off somewhere or it rotates the log files or whatever.

So in normal Go, I can make this log function do some slow IO, call `go log(message)`, and I'm set. Or I can make the goroutine created inside the log function when it's needed, either way.

With nurseries, where do I create a nursery? I don't want the code that calls log to have to wait for the logging to complete. So the caller shouldn't own a nursery and pass it to the logging library. The logging library can't create a nursery when it needs to do the occasional slow operation because the caller will have to wait for that too, if I understand correctly.

So, do I need to just use some other abstraction entirely, or how would I implement this with nurseries? This almost reminds me of using Java threadpools... you can more or less do anything you want to do with them, you just need to create lots of queues and consumer objects and it takes several abstractions to implement what could just be a single abstraction.

dragonwriter on March 19, 2021 | parent | next [–]

> With nurseries, where do I create a nursery

One option would be to have main() create and pass the nursery when it initializes the asynchronous logger.

main() would have to wait for all logging to complete, but nothing else would.

Spivak on March 19, 2021 | root | parent | next [–]

> main() would have to wait for all logging to complete

Which is presumably the desired behavior anyway as all objects fall out of scope during program termination.

geofft on March 19, 2021 | parent | prev | next [–]

https://stackoverflow.com/a/48300537

> One of Trio's weirdest, most controversial decisions is that it takes the position that the existence of a background task is not an implementation detail, and should *be exposed as part of your API. On balance I think this is the right decision, but it's definitely a bit experimental and has some trade-offs.*

There's some argument in there about why that's the case - the high-order argument, I think, is that when I'm done calling some function that uses your logging library, I might be surprised at some later point by seeing my code inside your logging setup. (Maybe the log server dropped the connection and now you're doing a CPU-intensive SSL handshake to reconnect to it, or something.)

I think the idiomatic way to do this would be that, yes, the code that calls log() should provide a nursery somehow, potentially by asking its own caller to provide a nursery. There's an example in that SO answer of a class that accepts a nursery in its constructor, plus a convenience wrapper that creates a nursery and uses the class once (and then waits on tasks to complete before returning).

If you really, really want the operation to happen in the background, there is https://trio.readthedocs.io/en/stable/reference-lowlevel.htm... , but it's bad style to go sticking random things there. At the very least, document this as part of your API.

jayd16 on March 19, 2021 | parent | prev | next [–]

You could implement the log calls as channel writes. Then you could have a long running log writer task read from the channel in a similar way to the accept() example?

Edit: After reading the docs it seems like you make a logging nursery and then your log calls throw tasks into that nursery?

neolog on March 19, 2021 | parent | prev | next [–]

Filesystem IO

https://trio.readthedocs.io/en/stable/reference-io.html#asyn...

eqvinox on March 19, 2021 | prev | next [–]

OpenMP's [0] #pragma parallel directives function very much in the same way. The following code:

```
#pragma omp parallel for
for (size_t i = 0; i < limit; i++)
  whatever();
```

fully encapsulates the workload distribution and rejoining. By the time the code after the loop executes, all parallel execution of the loop is done.

[0] https://en.wikipedia.org/wiki/OpenMP, earliest versions dating to 1997

Laremere on March 19, 2021 | parent | next [–]

There are important distinctions between this code and what it proposed by the OP. See the sections

https://vorpus.org/blog/notes-on-structured-concurrency-or-g...

and https://vorpus.org/blog/notes-on-structured-concurrency-or-g...

These differences elevate this proposal from handling parallel computation (something that is fairly easy to reason about with normal go routines anyways), to something that actually handles concurrency use cases in general.

eqvinox on March 19, 2021 | root | parent | next [–]

I was simply making an observation on what the control flow looks like on a source code level, nothing more and nothing less. OpenMP is, as you say, designed for computation, hence the directives it provides.

The control flow is still majorly different from go routines, and the argument for this source code expression is IMHO the quintessence of the article. There simply is less to reason about if you have these constructs. With a go routine you still need a join somewhere, which makes it "unstructured" like gotos.

int_19h on March 19, 2021 | prev | next [–]

> You can't break out of one function and into another, and a return can take you out of the current function, but no further. Whatever control flow shenanigans a function gets up to internally, other functions don't have to care.

Smalltalk, Ruby, and other languages in a similar vein beg to differ. And I don't think you could call e.g. Smalltalk "unstructured", either.

On the whole, I'm not really convinced by the arguments presented there. Most async code is effectively linear on any given code path, and thus you don't need any abstraction other than tasks/futures to synchronize. When multiple concurrent tasks are spawned, ideally, the facility that does so should bind them together in a single task that can then be awaited (or not, if necessary). This takes care of resource cleanup, error handling etc.

dragonwriter on March 19, 2021 | parent | next [–]

> Smalltalk, Ruby, and other languages in a similar vein beg to differ. And I don't think you could call e.g. Smalltalk "unstructured", either.

Smalltalk, Ruby, and any language with callcc is nearly as unstructured as it would be with goto (though most of them idiomatically minimize usage, which mitigates this in practice), which is also a part (implementation/optimization prospects are another) that Ruby banished continuations from the core language to an MRI-bundled library starting way back with 1.9, replacing the main uses with the more structured Fibers.

dan-robertson on March 19, 2021 | root | parent | next [–]

Callcc is pretty unstructured. Delimited continuations are a bit more structured and, to me, seem a useful primitive even if application code shouldn’t ever use them. Common Lisp has lexically scoped goto/return which can be useful. Example:

```
(defun foo-find (foo x &key (test #'eql))
  (foo-iter foo (lambda (bar)
    (bar-iter bar (lambda (y)
      (when (funcall test x y)
        (return-from foo-find y)))))))
```

Some people try to implement something similar using exceptions but this only works if nothing swallows your special exception on its way up to the block you’re trying to return from.

dragonwriter on March 19, 2021 | root | parent | next [–]

Sure, all of those (even callcc) are useful, and having them in an otherwise reasonably high-level language means you can use it to build plumbing in the language that would otherwise have to be primitives implemented in a lower-level language (e.g., the Common Lisp condition system, which is basically exceptions with superpowers, can be built fairly simply on top of call/cc; I once built a toy implementation for Ruby based on the one in *Practical Common Lisp*.)

I don't think having unstructured told in a language is a bad thing, though I think the core language / stdlib having the right structured tools (whether primitives or built in the language itself on top of the unstructured ones) that most code never needs to reach for the unstructured tools is desirable.

int_19h on March 24, 2021 | root | parent | prev | next [–]

I wasn't referring to call/cc tho, but rather to the ability of lambdas to "return" from the outer function/method. In Ruby, IIRC, you can also "break" etc.

This is completely orthogonal to call/cc, because it won't allow you to do this arbitrarily - you get an exception if there's nothing to return/break from anymore.

codeflo on March 19, 2021 | prev | next [–]

I think this advocates a certain style of concurrency rather than a particular language feature.

As hinted at in the article, nothing prevents me from spawning a global_nursery at the top of my entry point and making that visible everywhere. I can still use this global_nursery (or any other higher-level one) to break resource cleanup.

I do find the argument compelling. But Python is simply too powerful for this abstraction to work perfectly, you’d need further restrictions. Maybe something like linear types (or Rust’s borrow checker) to make sure local state doesn’t escape into global nurseries?

giltayar on March 19, 2021 | prev | next [–]

If anybody's interested, I built an implementation of Nurseries in JS about two years ago, and have been using it here and there:

https://www.npmjs.com/package/nursery

vvern on March 19, 2021 | prev | next [–]

This is timely. Cockroach has recently decided to lint against the go statement. We also don't like errgroup because its context propagation is implicit.

See https://github.com/cockroachdb/cockroach/pull/62243.

> lacker on March 19, 2021 | parent | next [–]
>
> Cockroach definitely seems like a case where you'd want to keep your design more conservative, putting in more effort and being stylistically more restrictive to reduce bugs and increase performance, compared to something like a stateless application that talks to many backend services and is not a big deal if it crashes occasionally.

dan-robertson on March 19, 2021 | prev | next [–]

This is somewhat similar to the idea of linear futures where a type system is used to ensure that each future is used exactly once. This still allows the kind of structure suggested in this article but with slightly different constraints. One issue is that linear futures don't really allow for a select-like operation which takes some futures for a value of type T and returns a single future for a clue of type T that becomes determined to the same value as on of the input futures (roughly the one that finishes first but slightly undefined).

A question for this proposal (or linear futures): how do you implement a time out? That is, a future that either becomes determined to some input future, or some error value if the input takes too long.

The other big issue with joining control flow in async systems is error handling: when you join two threads, you can get two successes, one error, or two errors. When you select between two threads, you can get a success *and*, later, an error. Typically you want to pass the first error on eagerly. But where to? Do you pass it to something depending on the result of the future, or something from the dynamic scope when the future was created, or something else? And what happens with the second error? It might be bad to ignore it but it might be too late to care also. And the place where you might want to pass the error could be long gone by now.

> NoodleIncident on March 19, 2021 | parent | next [–]
>
> I think the article handles both cases (timeouts and the first error) by cancelling the other running async operations as quickly as it can.

wskinner on March 19, 2021 | prev | next [–]

I was surprised to reach the end of the article and find that Trio is just a python library. Doesn't that mean that even if you use Trio, if you also use any other libraries, you still cannot be sure about the concurrency properties of your code unless you read the transitive closure of your dependencies? I expected Trio to be a new language which enforces nursery usage in the compiler.

> jdc0589 on March 19, 2021 | parent | next [–]
>
> I believe that is correct, unless it does some crazy monkey patching or something (which would be horrible).
>
> sounds just like some of the original problems with the python event-loop webservers: everything was great until you used something that wasn't great.

IHLayman on March 19, 2021 | prev | next [–]

I liked the article, but I don't understand how it handles interrupts? Wouldn't signal handling imply a global nursery? Another discussion discussed logging, which would also imply a global nursery. If a SIGHUP hits the process, how does the signal handling thread in the global nursery cleanly kill the logging thread?

I like the direction, but maybe there have been refinements in the last two years to handle these cases, because they seem unresolved/unresolvable.

jamesgreenleaf on March 19, 2021 | prev | next [–]

I considered making some joke about how "considered harmful" essays should be considered harmful, but then I found that I am twenty years late.

https://meyerweb.com/eric/comment/chech.html

> neolog on March 19, 2021 | parent | next [–]
>
> The name is a play on the classic "goto" essay.
>
> > ufo on March 19, 2021 | root | parent | next [–]
> >
> > More importantly, it's directly referencing the contents of Dijkstra's essay and arguing that "go" is bad for the same reasons that "goto" is. In most of the other considered harmful blog posts the title is just a meme.

yccs27 on March 19, 2021 | parent | prev | next [–]

Indeed. For most of the article, the conclusion could be "concurrency is hard" instead of "go is harmful".

chsasank on March 19, 2021 | prev | next [–]

See this for how it actually works in code: https://chsasank.github.io/concurrent-programming-trio-tutor...

notacoward on March 19, 2021 | prev | next [–]

This seems like a *very* minor twist on fork/join, which only gets mentioned in a very snarky footnote. IMO it would be better to present it as such, instead of creating new terminology.

Jtsummers on March 19, 2021 | parent | next [–]

Go to the section "*go* statement considered harmful" itself. Specifically the subheading "Go statements break automatic resource cleanup".

He explicitly mentions *join* in that section. Specifically, not all of the concurrency libraries or language constructs offer a join-equivalent, and when they do it isn't language mandated that it is used. It remains "unstructured" and requires, like with his analogy to goto, careful use and monitoring by the programmer rather than getting language/library behavior guarantees.

Laremere on March 19, 2021 | prev | next [–]

I started reading this article strongly disagreeing with it, but am convinced to the point that I would really like to try this in practice.

My initial disagreement was centered in that nurseries seemed overly optimized for the "do a bunch of processing in parallel and then return a result" use case. This seems nice, but not necessary as Go's channel and waitgroup privatives solve this easily enough.

My concerns were around use cases that I can initially think of, which don't fit this easy mold:

- Running servers, with connections handled on threads (eg, typical Go http.ListenAndServe). However this works with nurseries very well (when it doesn't with, eg, parallel do). I expect that if I have that server run, it will only return with an error or having been stopped, and that all the pending connections will have been finalized.

- Game engines: If I want to write a concurrent game engine, I don't want to be constantly creating and removing threads. Instead I want to spawn several well organized threads, some with their own tasks, others working from a shared pool. I want this processes to exist for the entirety of the time the game is running, so do I really need nurseries to manage cleanup and reuniting processes? That characteristic doesn't solve any significant problems. However, the statement that if you want to allow a function to run something in the background, you need to pass it an already opened nursery is convincing. This structures the main function to likely have a nursery created for the engine game engine process, and passed to the various systems which need to start background threads. This should make the important reasoning about carefully scheduling work clearer to look at and trace.

- I recently worked on a project where we had a large volume of queries on fast changing data, with reasonably limited total data requirements. The solution was to have a background thread do the following: 1. Collect all waiting requests. 2. Update the internal cache with the state of the entire system. 3. In parallel let the various requests run their queries on the cached state. 4. Wait for the queries to finish, before restarting with #1. This enforces that requests always get state which is ordered after the request started (so create then query won't have a race condition), and is easy to reason about as the state of the cache cleanly alternates between being written by one thread, and read by many threads.

Nurseries really don't help at all with this use case. However they don't harm it in any way either: The request threads are started by the server handling requests, which is blocking the main thread. The cache handling thread is started by the main thread before starting the server, which just works. The main thread keeps the nursery around, showing that if you looked at the main thread that a background task would be running there.

So, all in all, I look forward to trying this style of concurrency.

bionhoward on March 19, 2021 | prev | next [–]

I wrote one of these "considered harmful" papers about the way Terraform passes ID / ARN / NAME / thing_name, policy.id, yadda yadda etc etc (a different way to refer to each resource from each other resource yields n^2 different ways things could be wired up) and I regret it. Seems a bit negative and pointless and I wish I hadn't written it.

I'd rather just say, hey, if Terraform did "pass by reference" we could type less and implementers could take the data they need instead of devs needing to figure out which of 4 different IDs to pass for each of 4 different references for 20 different resources

TLDR: Dont write a considered harmful paper. It's a trap. Write about how to make stuff better, not about how stuff is bad!

Blikkentrekker on March 19, 2021 | prev | next [–]

How can this work:

```
asyncio.create_task(myfunc())               # Python with asyncio
```

Surely it would then call `myfunc` in the main thread, block until it be done, then return whatever value it returns if it return at all, and call `create_task` with that value in the event that it return?

pansa2 on March 19, 2021 | parent | next [–]

IIUC if `myfunc` is an async function, calling it will immediately return a coroutine object.

None of the function's code will run until it is `await`ed. (Unlike some other languages, async-await in Python is not "synchronous to the first await".)

incrudible on March 19, 2021 | parent | prev | next [–]

It is implied that myfunc returns an awaitable object, not its ultimate result.

eru on March 19, 2021 | prev | next [–]

I've used both `trio` and `asyncio` in Python recently.

Trio's nurseries seem to make error handling easier.

zinclozenge on March 19, 2021 | parent | next [–]

I like trio a lot, wish there was a grpc library with typing hints that could run on it.

eru on March 19, 2021 | root | parent | next [–]

Yes, that would be neat.

Trio's nurseries remind a little bit of IO in Haskell.

Forgetting all about Monads etc, you can re-interpret IO-typing in Haskell as the requirement to keep carry something like a 'nursery' around with your code. Parts of code that don't have access to such a 'nursery'-like object can't do IO. (And because it's Haskell, that access is also tracked in the types.)

samprotas on March 19, 2021 | root | parent | next [–]

It's also very directly related the to "effectively standard" (but not included) async library in Haskell.

https://hackage.haskell.org/package/async

The package description at the top of the link touches on the motivations which basically mirror this article.

I've personally never reached for the built in forkIO. withAsync or it's helpers like mapConcurrently are always equally capable, easier to use, and with none of the foot guns.

bradhe on March 19, 2021 | prev | next [–]

I think "considered harmful" is a little strongly worded. There's some conclusions the author jumps to to support their point explicitly in the titular section of the article.

dangerbird2 on March 19, 2021 | parent | next [–]

It appears to be more of a homage to "goto statement considered harmful" paper than it is a statement on the "harmfulness" of the `go` statement, as the author comparing `go` with `goto` and talking about Dijkstra's paper.

conradludgate on March 19, 2021 | prev | next [–]

Yet another comment about rust... :)

They mention RAII as an alternative to the `with` statement, and with Rust's borrow checker and strict memory management between async tasks, this isn't such a big problem there. A task *must* have a guarantee that any data they use will exist during the entire time the task needs it.

Async in rust is already a very contentious topic though, much like this. I do find `go` statements tricky to reason about sometimes, and even the question of where to put them. Should `program.Start()` spawn the goroutine itself, or should you be expected to handle that. I've used both patterns in my work, but communicating that is very tricky

thayne on March 20, 2021 | prev | next [–]

Are there libraries in other languages that work similarly to Trio?

Jtsummers on March 20, 2021 | parent | next [–]

https://trio.discourse.group/t/structured-concurrency-resour...

Has a short list of library implementations for C, Python, Swift, and Kotlin. As I understand it (not a Java programmer, well not up to date on the ecosystem) it's being adopted/explored in Java via Project Loom as well.

systemBuilder on March 19, 2021 | prev | next [–]

Just what's needed! Another code Nazi telling everybody the "right" way to program because they have seen it all, done it all, and now, finally, know it all!

At least he put in a library unlike the Go designers who welded their concurrency primitives in stone!

makkovjulia on March 19, 2021 | prev | next [–]

I do not think that's true.

rob74 on March 19, 2021 | prev [–]

If you use a Dijkstraesque title like this one, you should either be a recognized expert on the subject yourself or at least cite some articles that support your statement. Using it to promote your "better" concurrency library smells like clickbait...

EDIT: Ok, I stand corrected, this seems to be an influential article. But the title still smells like clickbait to me (especially because it singles out a well-known statement of a well-known language that has concurrency as one of its main selling points when it actually refers to using all types of threads - "green" or not)...

mrkeen on March 19, 2021 | parent | next [–]

> you should either be a recognized expert on the subject yourself

Did you just request the Argument from Authority fallacy?

> at least cite some articles that support your statement

This is the article. Others can cite it.

kasperni on March 19, 2021 | parent | prev [–]

Well, so far the article is probably the main reason why structured concurrency is being added to both Java and Kotlin.

platinumrad on March 19, 2021 | root | parent [–]

Yeah this random blog post from 2018 is definitely why Oracle started Project Loom in 2017.

kasperni on March 19, 2021 | root | parent [–]

I didn't say it started Project Loom I'm saying this article is one of the main reasons structured concurrency is being made part of project Loom.

A link to this particular article have literally been on every project Loom slide coming out of Oracle for the last two years. You can watch Ron Pressler talk about it himself here [1].

[1] https://www.youtube.com/watch?v=lIq-x_iI-kc&t=14m30s

Guidelines | FAQ | Lists | API | Security | Legal | Apply to YC | Contact

Search: